

FILESYSTEM

Sistemas de Archivos Modernos

1. Fundamentos de Fragmentación y Journaling

En los sistemas operativos modernos, la gestión eficiente del almacenamiento es crucial para garantizar un alto rendimiento. Dos fenómenos que impactan directamente en el rendimiento del sistema de archivos son la *fragmentación* y el *journaling*. Este capítulo se enfoca en la definición, las causas y los efectos de la fragmentación, así como en las técnicas de journaling utilizadas para mitigar los problemas derivados de fallos en el sistema.

1.1. Fragmentación

La *fragmentación* es un fenómeno que ocurre cuando los datos de un archivo se distribuyen de manera dispersa a través del espacio de almacenamiento disponible. La fragmentación puede presentarse de dos maneras:

a. Fragmentación Interna

La *fragmentación interna* ocurre cuando los archivos se almacenan en unidades de tamaño fijo, pero no se utiliza todo el espacio de almacenamiento asignado. Este fenómeno se da principalmente cuando los archivos son más pequeños que el bloque de datos asignado. La *fragmentación interna* es más común en sistemas con tamaños de bloque grandes, lo que ocurre típicamente en sistemas de archivos como FAT32.

Para calcular la fragmentación interna, se utiliza la siguiente fórmula:

$$W = \text{Tamaño de clúster} - (\text{Tamaño archivo} \bmod \text{Tamaño clúster})$$

Por ejemplo, si un archivo de 3 KB se almacena en un bloque de 4 KB, se desperdician 1 KB.

Esta fórmula sirve para sistemas de archivo *FAT* o *ext4* (Clúster/Bloque). En *ext4*, debido a la asignación de *extents* (bloques contiguos), la fragmentación interna puede ser menos significativa. Sin embargo, si se usan clústeres de tamaño fijo en una estructura de bloques, la fórmula sigue siendo válida.

Para calcular el desperdicio total en el sistema de archivos debido a fragmentación interna:

$$Total_frag = \sum_{i=1}^n (W_i)$$

Donde:

- n = número total de archivos pequeños.
- W_i = espacio desperdiciado por cada archivo

b. Fragmentación Externa

La *fragmentación externa* ocurre cuando los bloques de almacenamiento disponibles no son contiguos, lo que genera espacios vacíos entre los datos almacenados. Esta fragmentación es especialmente relevante en *discos duros (HDD)*, donde los datos se escriben de forma secuencial, y los bloques de almacenamiento no pueden reutilizarse fácilmente debido a la presencia de vacíos en el espacio físico.

Cuando un archivo se fragmenta externamente, el sistema operativo debe acceder a múltiples ubicaciones del disco, lo que reduce la velocidad de lectura y escritura. Esta fragmentación afecta el rendimiento general del sistema, ya que los cabezales del disco deben moverse a distintas ubicaciones físicas para acceder a todos los fragmentos del archivo

Si hay bloques libres de 2 KB dispersos en el disco y un archivo de 6 KB necesita ser almacenado, el sistema dividirá el archivo en tres fragmentos de 2 KB, generando múltiples operaciones de lectura y escritura para ensamblarlo posteriormente. Esto puede afectar el rendimiento debido al mayor tiempo de búsqueda y acceso.

Para calcular la fragmentación interna, se utiliza la siguiente fórmula:

$$\text{Fragmentacion Externa} = \frac{\text{Espacio Libre no contiguo}}{\text{Espacio Libre total}} \times 100\%$$

Sin embargo, en algunos casos la fragmentación externa impide directamente almacenar un archivo, incluso si existe suficiente espacio total. Por ejemplo:

Si un disco de 50 MB tiene 12 MB de espacio libre distribuidos en tres bloques de 4 MB, no sería posible almacenar un archivo de 5 MB porque el sistema de archivos requeriría un bloque contiguo de al menos 5 MB para almacenarlo de manera secuencial.

$$\text{Fragmentacion Externa} = \frac{12\text{MB} - 4\text{MB}}{12\text{MB}} \times 100\%$$

$$\text{Fragmentacion Externa} = \frac{8\text{MB}}{12\text{MB}} \times 100\% = 66.67\%$$

Esto significa que aproximadamente el 66.67% del espacio libre está fragmentado y no se puede usar para almacenar archivos grandes de manera eficiente.

El archivo necesita un bloque contiguo de 5 MB, pero el bloque libre más grande es de 4 MB. Este problema es más común en sistemas FAT32 y NTFS, donde el esquema de asignación de bloques es más rígido en comparación con sistemas modernos como ext4 o Btrfs, que manejan mejor la fragmentación y la reubicación de bloques

1.2. Journaling (Registro por Diario)

El *journaling* es una técnica utilizada en sistemas de archivos para mantener la *consistencia de datos* y evitar la pérdida de información en caso de fallos del sistema, como cortes de energía o errores de hardware. El objetivo principal del journaling es registrar todas las transacciones del sistema de archivos en un "diario" (log) antes de realizar cambios en los archivos o estructuras de datos del sistema.

Tipos de Journaling

Hay varios tipos de journaling que se implementan en diferentes sistemas de archivos, y su principal diferencia radica en el nivel de detalle con el que se registran las operaciones y el impacto en el rendimiento.

- Metadata-only Journaling

En este tipo de journaling, solo se registran los cambios en los metadatos (información sobre el archivo, como nombre, permisos y ubicación), mientras que los datos no se registran. Este tipo es más rápido, pero si ocurre un fallo antes de que los datos sean escritos en el disco, es posible que se pierdan.

El sistema *ext3* utiliza el *metadata-only journaling* por defecto, lo que significa que solo se almacenan las modificaciones a los metadatos y no a los datos del archivo. Esto mejora el rendimiento, pero tiene el riesgo de perder los datos si ocurre una interrupción durante la escritura.

- **Full Journaling**

En este tipo de journaling, se registran tanto los cambios en los datos como en los metadatos. Aunque esto aumenta la seguridad del sistema al garantizar la integridad total de los datos, tiene un mayor impacto en el rendimiento debido a que se realiza una doble escritura (primero al journal y luego al disco).

En *ext3*, el journaling completo es opcional. Esta modalidad asegura una mayor protección de los datos, pero puede afectar el rendimiento debido a la necesidad de registrar los cambios en dos lugares antes de completarse.

- **Write-Ahead Logging (WAL)**

El Write-Ahead Logging (WAL) es un enfoque utilizado por varios sistemas de archivos modernos, como *NTFS* y *Btrfs*. En este tipo de journaling, los cambios se escriben primero en el journal antes de ser aplicados al disco. Esto permite recuperar el estado anterior del sistema de archivos si ocurre un fallo. Proceso básico en WAL:

- La operación de escritura es registrada primero en el journal.
- Luego, la operación se aplica al sistema de archivos.
- Una vez confirmada, el registro se elimina del journal.

Si ocurre un fallo (como una caída de energía) antes de que la operación termine, el sistema operativo puede rehacer o deshacer la operación basándose en los registros del journal, garantizando la integridad del sistema.

En *NTFS*, se utiliza el *Write-Ahead Logging* para asegurar la consistencia de los datos. Este enfoque ayuda a que el sistema pueda recuperar información de manera eficiente después de un fallo.

2. Fragmentación y Journaling en Sistemas de Archivos

2.1. Sistemas de Archivos Clásicos

FAT32

Fragmentación Interna en FAT32

La fragmentación interna se genera cuando el tamaño del archivo es menor que el tamaño del clúster asignado. Por ejemplo, si el tamaño del clúster es de 8KB y un archivo solo ocupa 4KB, entonces los 4KB restantes no se pueden utilizar por otros archivos, lo que genera fragmentación interna.

Caso práctico: 5000 archivos de 1 KB cada uno en FAT32 con clúster de 32KB

$$W = 32KB - (1 KB \bmod 32KB)$$

$$W = 32KB - (1KB)$$

$$W = 31KB \text{ Por archivo}$$

$$Total_frag = 31KB \times 5000 = 151.37 MB$$

Fragmentación Externa en FAT32

La fragmentación externa en FAT32 ocurre cuando los archivos se almacenan en ubicaciones no contiguas en el disco. Esto puede suceder debido a la forma en que FAT32 maneja la asignación de bloques. FAT32 no tiene un mecanismo avanzado de asignación dinámica, lo que puede provocar una dispersión de los archivos a medida que se escriben y borran.

En el caso de un sistema FAT, si el disco tiene 100 MB de espacio libre y 40 MB de esos 100 MB están dispersos en bloques pequeños y no contiguos, la fórmula sería:

$$\text{Fragmentacion Externa} = \frac{40\text{MB}}{100\text{MB}} \times 100\% = 40\%$$

Esto indica que el 40% del espacio libre está fragmentado, dificultando la asignación eficiente de archivos.

Journaling en FAT32

FAT32 no implementa journaling de forma nativa, lo que significa que no tiene un registro de las transacciones antes de que se realicen, lo que lo hace susceptible a la corrupción de datos en caso de un corte de energía o fallo del sistema. Sin embargo, se pueden usar herramientas de recuperación de datos para tratar de recuperar la información de un sistema FAT32 en caso de fallo.

2.2. Sistemas de Archivos Modernos

NTFS

Fragmentación Interna en NTFS

La *fragmentación interna* también es un problema en NTFS, especialmente cuando los tamaños de clúster no se ajustan adecuadamente al tamaño de los archivos. En NTFS, los clústeres pueden ser de tamaños variados, dependiendo del formato y la configuración del disco.

NTFS también tiene la ventaja de realizar asignaciones dinámicas de espacio, lo que puede reducir la fragmentación interna en comparación con sistemas como FAT32, que utilizan tamaños de clúster fijos.

Fragmentación Externa en NTFS

Al igual que FAT32, NTFS puede sufrir *fragmentación externa* si los archivos no se distribuyen de manera eficiente. Sin embargo, NTFS tiene un *algoritmo más avanzado* para gestionar la distribución de archivos y bloques en el disco, lo que reduce la probabilidad de fragmentación externa en comparación con FAT32.

La *fragmentación externa* en NTFS se puede minimizar utilizando herramientas de *desfragmentación automática* que reorganizan los bloques de datos para que estén más cercanos entre sí. A pesar de esto, la fragmentación externa aún puede ocurrir en discos muy llenos o en discos con archivos de gran tamaño.

Journaling en NTFS

NTFS utiliza Write-Ahead Logging (WAL) para garantizar la consistencia del sistema de archivos.

Los cambios se registran primero en un archivo especial llamado *\$LogFile* antes de aplicarse al disco. Esto permite que NTFS restaure los cambios pendientes si ocurre un fallo del sistema. NTFS soporta tres modos de journaling:

- **Writeback** → Solo registra cambios en el journal, sin asegurar que los datos se escriban inmediatamente.
- **Ordered** → Los cambios se escriben en el journal y en el disco de manera ordenada.
- **Full** → Registra tanto los cambios como los datos, garantizando la máxima integridad pero con mayor impacto en el rendimiento.

Optimizaciones en NTFS

NTFS también introduce varias optimizaciones para mejorar la eficiencia y reducir la fragmentación:

- **Asignación basada en clústeres:** NTFS organiza los archivos mediante clústeres (bloques de tamaño fijo) y utiliza una tabla llamada **Master File Table (MFT)** para rastrear los archivos. Esto permite una recuperación rápida de datos y facilita la reasignación de bloques.
- **Sparse Files:** NTFS permite la creación de archivos dispersos (sparse), donde solo los bloques que contienen datos reales se asignan en disco, reduciendo el uso innecesario de espacio y la fragmentación interna.
- **Defragmentación Automática:** NTFS puede ejecutar automáticamente procesos de desfragmentación en segundo plano mediante herramientas como "*defrag*" o el servicio de optimización de disco de Windows, lo que reduce la fragmentación externa.
- **Delayed Allocation (Buffering):** Similar a EXT4, NTFS permite retrasar la asignación de bloques para escribirlos de manera contigua, lo que reduce la fragmentación externa y mejora el rendimiento de escritura secuencial.

Consideremos el siguiente ejercicio práctico para calcular la fragmentación externa en un sistema NTFS:

- Un archivo de 500 MB está fragmentado en 5 fragmentos. El sistema tiene 50 MB de espacio libre disperso en bloques de 10 MB.
- Hay 15 MB de espacio perdido debido a la distribución no contigua de los bloques.

La fragmentación externa se calcula así:

$$\text{Fragmentacion Externa} = \frac{\text{Espacio perdido}}{\text{Tamaño total del archivo}} \times 100\%$$

$$\text{Fragmentacion Externa} = \frac{15\text{MB}}{500\text{MB}} \times 100\% = 3\%$$

La fragmentación externa en este caso es de 3%, lo que indica que NTFS está gestionando la fragmentación de manera razonable mediante la asignación de clústeres y el uso de buffering.

EXT4

A diferencia de sus predecesores (EXT3), EXT4 introduce mejoras significativas tanto en rendimiento como en fiabilidad. Aunque EXT4 está diseñado para minimizar la fragmentación a través de optimizaciones como el *extents-based allocation*, en ciertos escenarios, especialmente en discos con gran cantidad de escritura, puede seguir ocurriendo fragmentación.

Fragmentación Interna en EXT4

La *fragmentación interna* ocurre cuando un bloque de datos está parcialmente vacío, desperdiciando espacio. Esto puede ser más frecuente cuando se utilizan tamaños de bloque pequeños, lo cual es gestionado en parte por las optimizaciones de EXT4, aunque sigue siendo un problema potencial

Fragmentación Externa en EXT4

La *fragmentación externa* ocurre cuando los bloques de datos del archivo se distribuyen de manera no contigua. EXT4 gestiona esto mediante el uso de *extents* (extensiones de bloques), lo que ayuda a reducir la fragmentación externa, ya que los archivos ahora se asignan a bloques contiguos más grandes.

Journaling en EXT4

EXT4 utiliza el sistema de journaling para garantizar la integridad de los datos y mitigar la fragmentación, pero su uso puede afectar el rendimiento y aumentar la fragmentación interna.

- **Journaling de Metadatos:** En EXT4, los metadatos (como las tablas de inodos) se registran en el journal antes de que los cambios se reflejen en el disco. Esto permite que el sistema de archivos se recupere rápidamente en caso de fallo.
- **Journaling Completo (Data Journaling):** También soporta journaling completo, donde tanto los metadatos como los datos se registran antes de aplicarse al disco. Esto mejora la integridad del sistema, pero puede aumentar la fragmentación interna debido a la doble escritura

Optimizaciones en EXT4

EXT4 introduce varias optimizaciones que ayudan a reducir la fragmentación y mejorar el rendimiento del sistema de archivos:

- **Extents:** Este mecanismo mejora la asignación de bloques, donde un único "*extent*" puede representar una secuencia continua de bloques de datos. Esto reduce la necesidad de almacenar múltiples entradas en las tablas de bloques y disminuye la fragmentación externa.
- **Delayed Allocation:** En EXT4, los bloques no se asignan inmediatamente cuando un archivo es escrito. En cambio, se retrasan hasta que sea necesario. Esta optimización reduce la fragmentación externa porque permite al sistema escribir los datos de manera contigua cuando el espacio está disponible.
- **Pre-allocation de Espacio:** Cuando se crean archivos grandes, EXT4 permite preasignar espacio, reduciendo la fragmentación interna y garantizando que el espacio esté disponible cuando sea necesario

Consideremos el siguiente ejercicio práctico para calcular la fragmentación externa en un sistema EXT4.

- Un archivo de 1GB está fragmentado en 4 *extents*.
- Cada extent tiene 256MB, pero los bloques se distribuyen en diferentes partes del disco.
- Hay un total de 30 MB de espacio desperdiciado debido a la distribución no contigua de los bloques

La fragmentación externa se puede calcular como el espacio desperdiciado debido a los bloques no contiguos:

$$\text{Fragmentación Externa} = \left(\frac{30MB}{1024MB} \right) \times 100 = 2.93\%$$

La fragmentación externa en este caso es aproximadamente 2.93%, lo que indica una baja fragmentación debido al uso eficiente de extents

XFS

XFS es un sistema de archivos de alto rendimiento diseñado para escalabilidad en servidores de gran capacidad y sistemas de almacenamiento avanzados. Aunque XFS usa un modelo de asignación basado en bloques similar al de EXT4, tiene características únicas para gestionar la fragmentación.

Fragmentación Interna en XFS

La fragmentación interna es menos frecuente en **XFS** debido a su algoritmo de asignación de bloques basado en el principio de *extents*.

Fragmentación Externa en XFS

XFS gestiona la fragmentación externa mediante el uso de una asignación de bloques dinámica, que garantiza que los bloques de un archivo estén distribuidos de manera eficiente en todo el disco. Sin embargo, en discos de gran capacidad y con grandes cantidades de datos, puede haber algo de fragmentación externa.

Journaling en XFS

XFS utiliza un sistema de journaling para registrar los cambios en los metadatos, lo que facilita la recuperación del sistema después de un fallo. XFS usa un *journaling de metadatos* para asegurar que los cambios en los metadatos, como la tabla de inodos y las asignaciones de bloques, sean coherentes. Aunque no utiliza journaling de datos (a diferencia de EXT4 en su modalidad de journaling completo), es extremadamente eficiente en términos de rendimiento.

BTRFS

Btrfs es un sistema de archivos de última generación que ofrece funciones avanzadas como la gestión de volúmenes, snapshots y un modelo de datos basado en copy-on-write (COW). **Btrfs** está diseñado para manejar grandes volúmenes de datos con un enfoque en la flexibilidad y la integridad.

Fragmentación Interna

Como Btrfs utiliza *extents* y tiene un esquema dinámico de asignación de bloques, la fragmentación interna es mínima, aunque puede ocurrir dependiendo del patrón de escritura.

Fragmentación Externa

Btrfs tiene un enfoque único en la fragmentación externa, dado que utiliza un esquema de asignación de bloques dinámico que puede permitir cierta fragmentación externa, pero su modelo de COW ayuda a minimizar este problema.

Journaling en Btrfs

Al igual que XFS, Btrfs usa un sistema de journaling, pero con algunas diferencias clave. Utiliza un journaling de metadatos combinado con el enfoque **COW** (Copy-On-Write), lo que permite una recuperación rápida en caso de fallos.

3. Hardware y Técnicas de Mitigación

En este punto se analiza en la **fragmentación** y cómo afecta a los discos duros tradicionales (HDDs) y las unidades de estado sólido (SSDs). Aunque ambos tipos de almacenamiento son utilizados ampliamente, presentan diferencias significativas en cómo manejan la fragmentación

3.1. Fragmentación en HDD

En un *disco duro (HDD)*, los datos se almacenan en platos giratorios que se mueven físicamente para acceder a las áreas de almacenamiento. Debido a esto, la fragmentación externa puede afectar directamente el rendimiento de la unidad.

Fragmentación Externa en HDD: En un HDD, los archivos se escriben en bloques de datos dispersos a lo largo de los platos. Cuando un archivo se divide en fragmentos no contiguos, el cabezal de lectura/escritura tiene que moverse más, lo que reduce la velocidad de acceso. Este proceso implica una sobrecarga mecánica significativa que degrada el rendimiento general.

Fragmentación Interna en HDD: Aunque la fragmentación interna en un HDD no es tan común como la fragmentación externa, puede ocurrir cuando se asignan bloques que no se llenan completamente, lo que conduce a un desperdicio de espacio.

3.2. Fragmentación en SSD

Por otro lado, en un *SSD*, que utiliza memorias flash para almacenar datos, la fragmentación interna y externa tiene un impacto muy diferente.

Fragmentación Externa en SSD: Aunque la fragmentación externa no afecta tanto a los SSDs como a los HDDs debido a la velocidad de acceso aleatorio rápida (sin necesidad de movimiento mecánico), puede haber un pequeño impacto en el rendimiento de lectura en discos con una gran cantidad de archivos pequeños.

Fragmentación Interna en SSD: Los *SSDs* son más susceptibles a la *fragmentación interna* debido a la manera en que los datos se gestionan en bloques. Dado que los bloques de memoria flash tienen un tamaño fijo y no pueden reescribirse directamente, los archivos pequeños que no llenan completamente un bloque pueden desperdiciar espacio en el SSD.

En SSDs, este fenómeno puede generar *bloques parcialmente llenos* que no pueden reutilizarse hasta que se haya vaciado el bloque completo, lo que reduce la eficiencia del almacenamiento.

3.3. Desfragmentación

La *desfragmentación* es un proceso que reorganiza los datos en el disco para reducir tanto la *fragmentación externa* como la *fragmentación interna*, asegurando que los archivos se almacenen de manera contigua. Este proceso varía dependiendo del tipo de sistema de almacenamiento, y su efectividad depende del medio utilizado.

Desfragmentación en HDD

En los discos duros tradicionales (HDD), la desfragmentación es un proceso muy eficaz para mejorar el rendimiento, ya que reduce los tiempos de acceso a los datos. Dado que los HDDs tienen partes móviles (el *cabezal de lectura/escritura*), la fragmentación aumenta los tiempos necesarios para acceder a archivos dispersos por diferentes sectores del disco. Al mover los fragmentos dispersos a bloques contiguos, el cabezal puede acceder más rápidamente a los datos, mejorando el rendimiento general.

Si un archivo de 1GB estaba fragmentado en 10 fragmentos de 100MB, la desfragmentación podría reorganizar esos fragmentos de manera contigua, lo que permitiría que el cabezal acceda al archivo sin desplazarse innecesariamente entre diferentes fragmentos. Por ejemplo, si el tiempo de acceso promedio para un fragmento es de 0.5 ms, al tener 10 fragmentos, el tiempo total de acceso sería de 5 ms. Después de la desfragmentación, el archivo estaría contiguo, reduciendo el tiempo de acceso a 0.5 ms para el archivo completo.

Desfragmentación en SSD

En los *SSDs*, la desfragmentación no es tan efectiva como en los *HDDs* debido a la naturaleza de la memoria flash. Los *SSDs* tienen tiempos de acceso extremadamente rápidos y no tienen partes móviles, lo que hace que la fragmentación no afecte significativamente el rendimiento. Además, la desfragmentación de un *SSD* puede generar más escrituras innecesarias, lo que puede reducir la vida útil del dispositivo debido al número limitado de ciclos de escritura de las celdas de memoria flash.

Los *SSDs* tienen un número limitado de ciclos de escritura (usualmente en el rango de miles de ciclos). Cada vez que se realiza una operación de desfragmentación, se escriben grandes cantidades de datos en las celdas de memoria, lo que aumenta el desgaste de las celdas. Esto puede reducir la vida útil del *SSD*. Por ejemplo, si un *SSD* tiene un total de 1,000 ciclos de escritura por celda, y cada operación de desfragmentación realiza 100 GB de escrituras en total, es posible que las celdas de memoria se desgasten más rápido.

3.4. Técnicas para reducir la fragmentación

Uso de Clústeres Grandes

El uso de *clústeres grandes* es una técnica que puede reducir la *fragmentación interna*, especialmente en sistemas de archivos que almacenan muchos archivos pequeños. Al utilizar clústeres más grandes, se reduce la cantidad de fragmentación interna, ya que los archivos pequeños se almacenan en bloques más grandes, lo que reduce la cantidad de espacio desperdiciado.

Utilizar clústeres más grandes puede ayudar a reducir la fragmentación interna, ya que se almacenan más datos dentro de un clúster y se desperdicia menos espacio. Sin embargo, este enfoque puede generar más *fragmentación externa* si no se maneja correctamente el espacio libre disponible, ya que es más probable que los clústeres grandes se fragmenten si hay muchos archivos pequeños.

Si un archivo de 1MB se guarda en un clúster de 4MB, el 75% del clúster queda vacío. Esto produce una fragmentación interna del 75%. Sin embargo, la fragmentación externa puede disminuir si los archivos grandes se almacenan en bloques contiguos de clústeres grandes.

Wear Leveling en SSDs

El *wear leveling* es una técnica utilizada en *SSDs* para distribuir uniformemente las escrituras a través de las celdas de memoria, evitando que algunas celdas se desgasten más rápidamente que otras. El objetivo de esta técnica es prolongar la vida útil del *SSD* y reducir el riesgo de fallos prematuros.

Principio de **wear leveling**; los *SSDs* emplean algoritmos de *wear leveling* que aseguran que todas las celdas de memoria se reescriban de manera uniforme. Esto impide que ciertas celdas se desgasten prematuramente, lo que optimiza el rendimiento y previene la fragmentación interna al distribuir las escrituras de manera más equitativa.

TRIM en SSDs

El comando *TRIM* es utilizado en *SSDs* para informar al sistema operativo que ciertos bloques de datos ya no están en uso y pueden ser borrados y reutilizados. Este comando ayuda a mantener la eficiencia del *SSD*, evitando la acumulación de datos obsoletos o fragmentados, y reduce la fragmentación interna.

Cuando el sistema operativo envía un comando *TRIM*, le dice al *SSD* que ciertos bloques de memoria ya no se necesitan y pueden ser limpiados. Esto evita que el *SSD* acumule datos fragmentados, mejorando la eficiencia de la escritura y reduciendo la fragmentación interna.

Ejercicios

Fragmentación Interna y Externa

1. Un sistema FAT16 tiene las siguientes configuraciones: Tamaño de bloque= 512 bytes y Clúster = 8 bloques

Preguntas:

- a. ¿Cuál es el tamaño de clúster en bytes?
 - b. Si un archivo de 10.5 KB está almacenado de manera fragmentada en clústeres no contiguos, ¿cuántas entradas en la tabla FAT se utilizarán?
 - c. ¿Cuánto espacio desperdiciado habría debido a fragmentación interna?
2. Un sistema ext4 tiene: Tamaño de bloque: 4 KB y Bitmap de bloques: 0xFFFF0000FFFF0000 (hex).

Preguntas:

- a. ¿Cuántos bloques libres hay en este grupo?
- b. Si se asigna un archivo de 20 MB, ¿cuántos bloques ocupará?
- c. Calcule la fragmentación interna si el archivo tiene un tamaño real de 20.5 MB.
- d. Calcule la fragmentación interna si el archivo tiene un tamaño real de 20.3 MB.

Bibliografía

- Silberschatz, A., Galvin, P., & Gagne, G. (2018). *Operating System Concepts* (10th ed.).
- Tanenbaum, A. (2015). *Modern Operating Systems* (4th ed.).
- Love, R. (2010). *Linux Kernel Development* (3rd ed.).
- Bovet, D., & Cesati, M. (2005). *Understanding the Linux Kernel* (3rd ed.).